

一、课程总体要求与个人模块要求

本方向围绕“推理 / 强化学习 / 对齐”展开，要求学生以小组形式完成一个完整系统，同时每位同学必须负责一个能够独立运行、独立演示、独立讲解的核心模块。

1.1 总体要求

- 每位同学必须认领至少一个核心模块，可以两位同学共同负责一个模块。
- 每个核心模块必须能够独立运行、独立演示、独立讲解。
- 每组建议由 4-5 名同学组成，并形成完整系统。
- 组内模块之间必须存在明确的接口关系和协作关系。
- 最终验收时既看小组系统，也严格检查个人模块。
- 评测、错误分析、训练前后对比、运行日志等属于全组公共要求，不建议单独作为个人核心模块。

1.2 个人模块要求

要求	说明
独立输入输出	明确模块输入和输出，例如输入 prompt，输出模型回答；输入任务描述，输出调用轨迹。
独立运行	必须有自己的脚本、Notebook、CLI 命令或 Web Demo。
独立演示	验收时本人需要亲自运行、讲解并展示结果，可使用 vibe coding 制作演示系统。
独立代码验收	需要提交个人负责部分的代码、README、配置文件、运行日志、截图或视频。
可集成	需要提供函数、API、JSON 文件、命令行或中间文件接口给组内其他模块调用。
有实验或对比	至少包含 baseline、训练前后对比、不同参数对比、错误分析或案例分析。

二、环境配置与项目目录

2.1 基础环境

本项目默认使用已经配置好的 assignment_A Conda 环境。所有命令建议在项目根目录执行。

```
conda activate assignment_A
cd /data/yekaiyang/zjx/20260617/assignment_A
```

检查 GPU：

```
python -c "import torch; print(torch.__version__); print(torch.cuda.is_available());  
print(torch.cuda.device_count())"
```

如遇 torchvision 或 torchaudio 版本不匹配，需要保证 torch / torchvision / torchaudio 与 CUDA 版本一致。例如当前环境 torch 为 2.5.1+cu124 时，建议使用 torchvision==0.20.1、torchaudio==2.5.1。

2.2 目录说明

目录或文件	说明
Qwen1.5-0.5B-Chat	本地基础模型。
py-dpo-v0.1/py-dpo.parquet	Python 代码任务偏好数据原始文件。
dpo	偏好数据构造、DPO 训练、MBPP 评测代码和输出。
tts	推理增强、CoT prompt、CoT 评测相关代码。
LlamaFactory	训练框架源码。
2026 实训 A 方向.pptx	课程 PPT。

三、推理指令数据构造

3.1 模块目标

A1 模块负责把原始 Python 代码指令数据整理为 LLaMA-Factory 可训练的 instruction / input / output 格式，作为后续 SFT 微调的基础数据。它的作用是把原始任务文本变成模型可学习的监督样本，并进行必要的清洗和筛选工作。数据集的构造质量直接影响到模型的训练和测试效果。

3.2 输入数据与目标

项目	内容
原始数据	python_code_instructions_18k_alpaca 中的 parquet 文件
任务类型	Python 代码指令生成
目标格式	instruction / input / output
主要用途	供 SFT 训练以及后续推理验证
关键脚本	sft/scripts/prepare_code_sft_data.py

3.3 运行命令

执行数据准备：

```
cd /data/yekaiyang/zjx/20260617/assignment_A
```

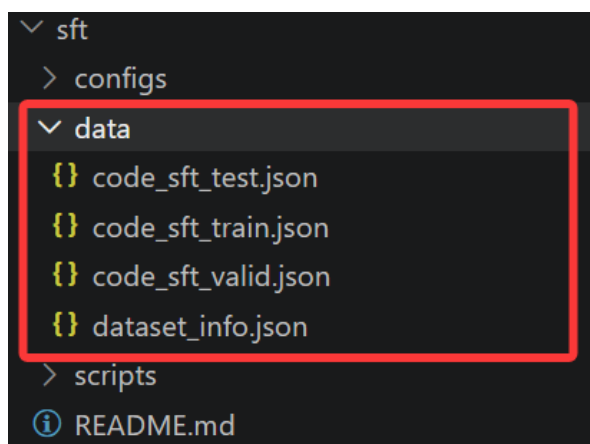
```
bash sft/scripts/prepare_data.sh
```

3.4 输出文件

文件	说明
sft/data/code_sft_train.json	用于 SFT 的训练集。
sft/data/code_sft_valid.json	用于监控训练效果的验证集。
sft/data/code_sft_test.json	用于最终推理评测的测试集。
sft/data/dataset_info.json	LLaMA-Factory 数据注册信息。

3.5 代码运行效果截图：

```
Read 18612 raw rows from 1 parquet file(s).
Kept 18612 valid unique rows.
Wrote train: 16750 -> sft/data/code_sft_train.json
Wrote valid: 930 -> sft/data/code_sft_valid.json
Wrote test : 932 -> sft/data/code_sft_test.json
Wrote registry -> sft/data/dataset_info.json
```



3.6 本节实训进阶目标

四、SFT 微调

4.1 模块目标

A2 模块利用 A1 生成的监督数据，在本地 Qwen1.5-0.5B-Chat 上进行全量 SFT。目标是让模型学会根据指令生成更准确、更符合 Python 代码风格的回答。

4.2 模型与配置

项目	内容
基础模型	./Qwen1.5-0.5B-Chat
训练框架	LLaMA-Factory
训练数据	sft/data/code_sft_train.json
验证数据	sft/data/code_sft_valid.json
训练配置	sft/configs/qwen15_code_full_sft.yaml
输出目录	sft/outputs/qwen15_code_full_sft

4.3 运行命令

启动训练：

```
cd /data/yekaiyang/zjx/20260617/assignment_A
bash sft/scripts/train.sh
```

若指定 GPU，则改为：

```
GPU_ID=0 bash sft/scripts/train.sh
```

批量预测测试集：

```
MODEL_PATH=your_trained_model_path \
bash sft/scripts/predict_full.sh
```

执行评测：

```
bash sft/scripts/evaluate_full.sh
```

4.4 输出结果

输出	说明
sft/outputs/qwen15_code_full_sft	训练得到的模型目录。
sft/outputs/qwen15_code_full_predict/generated_predictions.jsonl	测试集生成结果。

五、偏好数据构造

5.1 模块目标

本模块负责构造 DPO 或其他偏好学习算法所需的 chosen / rejected 数据，是后续对齐训练的关键输入。数据质量会直接影响 DPO 训练稳定性和模型输出质量。

5.2 原始数据说明

项目	内容
数据集地址	https://huggingface.co/datasets/jondurbin/py-dpo-v0.1
本地路径	py-dpo-v0.1/py-dpo.parquet
数据类型	Apache Parquet
任务方向	Python coding DPO 偏好数据
核心字段	prompt、chosen、rejected、id

原始数据是典型的 DPO 三元组：同一个 prompt 下，chosen 是更优代码回答，rejected 是较差代码回答。DPO 训练不需要人工分数，而是学习 chosen 相对 rejected 更优的偏好关系。

```
{
  "id": "样本 id",
  "prompt": "Python 编程任务描述",
  "chosen": "更优的 Python 代码回答",
  "rejected": "较差的 Python 代码回答"
}
```

prompt	chosen	rejected	id
string · lengths 41e513 58.7%	string · lengths 48e972 30.1%	string · lengths 2e3.24k 92.4%	string · lengths 36
Use the function to debug the given program and prevent the segmentation fault. Your solution...	One possible solution to prevent the segmentation fault and handle the cas...	def debug_program(arr): n = len(arr) left = 0 right = n - 1 while left <= right: mid = (left + right) // 2 if debug_program(arr[mid]): left = mid + 1 else: right = mid return right	8c94: 5f8c
Write an algorithm in Python to determine if a number is prime or composite. Your algorithm...	Here is the algorithm to determine if a number is prime or composite with a...	def is_prime(n): # Check for edge cases if n <= 1: return False elif n == 2: return True for i in range(3, int(n**0.5) + 1): if n % i == 0: return False return True	9d79: 5089
Compose a function named average_list which takes a list of numbers as its argument and...	Here is the implementation of the average_list function: ```python	def average_list(lst): if not isinstance(lst, list): raise ValueError('Input must be a list') return sum(lst) / len(lst)	01a2: 54f7
Rewritten Test: Write a function in Python that deletes a particular row in an SQLite database...	Here is a possible implementation of the function that meets the given...	Here is a function that deletes a particular row in an SQLite database...	3dfc: 524e
Write a program that extracts all the email addresses from a given string. The program...	Here is a corrected implementation of the 'extract_emails' function that...	```python import re def extract_emails(string): # Corrected implementation return re.findall(r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}', string)	776d: 52e6
Using the following code, clean up the bugs and make sure the output functions correctly...	To fix the bugs and implement the sorting algorithm, you can use the...	Here's a possible implementation of the sorting algorithm: ```python def bubble_sort(arr): n = len(arr) for i in range(n): for j in range(1, n-i): if arr[j] < arr[j+1]: arr[j], arr[j+1] = arr[j+1], arr[j] return arr	d4d9: 52fd

Huggingface 数据结构截图

5.3 本节实训基础要求（可以在当前代码的引导下完全实现）

1. 读取原始 py-dpo.parquet 数据文件，提取其中的 prompt、chosen、rejected 等字段。
2. 对原始样本进行清洗，去除缺失 prompt、chosen 或 rejected 的无效样本。

3. 将原始偏好数据转换为 LLaMA-Factory 支持的 DPO ranking 数据格式。
4. 按照随机种子划分训练集和测试集，并分别保存为 `code_dpo_train.json` 和 `code_dpo_test.json`。
5. 自动生成 `dataset_info.json`，完成数据集在 LLaMA-Factory 中的注册。
6. 输出数据处理日志，包括读取样本数、训练集数量、测试集数量和输出文件路径。

5.4 处理流程

1. 读取原始 parquet 文件：py-dpo-v0.1/py-dpo.parquet。
2. 使用随机种子打乱数据，默认 seed=42。
3. 清洗无效样本，保留 prompt、chosen、rejected 都非空的数据。
4. 默认抽取 TEST_SIZE=500 条作为测试集，其余样本作为训练集。
5. 训练集转换为 instruction / input / chosen / rejected 格式。
6. 测试集转换为 instruction / input / output 格式，output 使用原始 chosen。
7. 写出 `dataset_info.json`，将 `code_dpo_train` 注册为 ranking 数据集。

训练集格式：

```
{
  "instruction": "Complete the following Python coding task. ...",
  "input": "",
  "chosen": "better Python solution",
  "rejected": "worse Python solution"
}
```

测试集格式：

```
{
  "instruction": "Complete the following Python coding task. ...",
  "input": "",
  "output": "reference Python solution"
}
```

5.5 代码运行命令

运行数据准备：

```
bash dpo/scripts/prepare_data.sh
```

小样本调试：

```
MAX_TRAIN_SAMPLES=200 TEST_SIZE=50 bash dpo/scripts/prepare_data.sh
```

自定义随机种子：


```
SEED=123 bash dpo/scripts/prepare_data.sh
```

自定义原始数据目录和输出目录：

```
SOURCE_DIR=py-dpo-v0.1 OUTPUT_DIR=dpo/data bash dpo/scripts/prepare_data.sh
```

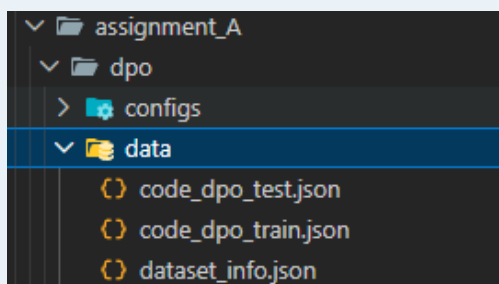
核心 Python 脚本也可以直接运行：

```
python3 dpo/scripts/prepare_dpo_data.py \  
    --source_dir py-dpo-v0.1 \  
    --output_dir dpo/data \  
    --test_size 500 \  
    --seed 42 \  
    --max_train_samples 0
```

5.6 代码运行图例

```
(assignment_A) yekaiyang@nlp1ab:~/zjx/20260617/assignment_A$ bash dpo/scripts/prepare_data.sh  
Read 9466 raw rows from 1 parquet file(s).  
Wrote train: 8924 -> dpo/data/code_dpo_train.json  
Wrote test : 485 -> dpo/data/code_dpo_test.json  
Wrote registry -> dpo/data/dataset_info.json
```

数据预处理的输出



文件保存的位置

```
code_dpo_test.json X  
zjx > 20260617 > assignment_A > dpo > data > code_dpo_test.json  
1  [  
2  {  
3    "instruction": "Complete the following Python coding task. Return a correct, readable Python :  
4    "input": "",  
5    "output": "Here's an implementation of an iterator in Python that yields prime numbers within  
6  },  
code_dpo_train.json X  
zjx > 20260617 > assignment_A > dpo > data > code_dpo_train.json  
1  [  
2  {  
3    "instruction": "Complete the following Python coding task. Return a correct, readable Python  
4    "input": "",  
5    "chosen": "1. Initialize an empty set to store the distinct words.\n2. Convert the given sent  
6    "rejected": "def word_count(sentence):\n    # Remove punctuation marks and convert to lowerc  
7  },
```

训练和测试集文件示例

当前项目已生成的数据规模：训练集 8924 条，测试集 485 条，总计 9409 条有效样本。

5.7 输出文件

文件	说明
----	----

dpo/data/code_dpo_train.json	DPO 训练集，包含 instruction / input / chosen / rejected。
dpo/data/code_dpo_test.json	生成测试集，包含 instruction / input / output。
dpo/data/dataset_info.json	LLaMA-Factory 数据注册表。

5.8 本节实训进阶要求

1. 修改 prepare_dpo_data.py，增加数据统计功能，例如输出平均 prompt 长度、chosen 长度、rejected 长度。
2. 增加数据质量过滤逻辑，例如过滤掉 chosen 或 rejected 太短的样本。
3. 增加命令行参数 --min_response_len，用于控制最短回答长度。
4. 将训练集和测试集划分方式从固定数量改为按比例划分，例如 95% 训练、5% 测试。
5. 增加 sample_preview.json，保存前 10 条转换后样本，方便人工检查。

六、DPO 对齐训练

6.1 模块目标

本模块使用上一模块生成的偏好数据进行 DPO 对齐训练，使模型在 Python 代码任务上更倾向于生成 chosen 风格的高质量答案。

6.2 模型与训练数据

项目	内容
基础模型	assignment_A/Qwen1.5-0.5B-Chat
训练框架	LLaMA-Factory
训练数据	dpo/data/code_dpo_train.json
训练配置	dpo/configs/qwen15_code_full_dpo.yaml
训练输出	dpo/outputs/qwen15_code_full_dpo

code_dpo_train.json

260617 > assignment_A > dpo > data > code_dpo_train.json

```
{
  "instruction": "Complete the following Python coding task. Return a correct, readable Python",
  "input": "",
  "chosen": "1. Initialize an empty set to store the distinct words.\n2. Convert the given sent",
  "rejected": "def word_count(sentence):\n    # Remove punctuation marks and convert to lowerc",
},
{
  "instruction": "Complete the following Python coding task. Return a correct, readable Python",
  "input": "",
  "chosen": "Sure, here's the updated calculation:\nThe sum of the first 10 consecutive odd nat",
  "rejected": "Sure, here's the code to perform the calculation in Python:\n\npython\n# Func",
},

```

dataset_info.json

260617 > assignment_A > dpo > data > dataset_info.json

```
{
  "code_dpo_train": {
    "file_name": "code_dpo_train.json",
    "ranking": true,
    "columns": {
      "prompt": "instruction",
      "query": "input",
      "chosen": "chosen",
      "rejected": "rejected"
    }
  },
  "code_dpo_test": {
    "file_name": "code_dpo_test.json"
  }
}
```

训练数据格式和注册表

6.3 本节实训基础要求（可以在当前代码的引导下完全实现）

1. 基于已经构造好的 code_dpo_train.json，配置 DPO 训练所需的数据路径和数据数据集名称。
2. 加载本地基础模型 Qwen1.5-0.5B-Chat，并将其同时作为训练初始模型和参考模型。
3. 使用 LLaMA-Factory 启动 DPO 训练流程，完成 chosen / rejected 偏好对齐训练。
4. 配置训练过程中的关键参数，包括训练轮数、学习率、batch size、梯度累积和最大输入长度。
5. 支持通过环境变量指定 GPU、Python 解释器和训练配置文件。
6. 将训练后的模型、tokenizer、训练状态和日志保存到指定输出目录。
7. 使用给出的 MBPP 评估脚本或公开 benchmark 评估模型性能。

6.4 关键配置说明

参数	值	说明
model_name_or_path	./Qwen1.5-0.5B-Chat	policy 初始模型。
stage	dpo	使用 DPO 训练阶段。
finetuning_type	full	全参数微调。

pref_beta	0.1	DPO 偏好约束强度。
pref_loss	sigmoid	标准 DPO 损失。
ref_model	./Qwen1.5-0.5BChat	reference model，与 policy 同一点。
dataset	code_dpo_train	A3 生成并注册的数据集。
cutoff_len	2048	最大输入长度。
per_device_train_batch_size	1	单卡 batch size。
gradient_accumulation_steps	8	梯度累积步数。
num_train_epochs	3.0	训练轮数。
plot_loss	true	训练结束后保存 loss 曲线。

注意：DPO 会同时加载 policy model 和 reference model，显存占用通常高于 SFT。
如遇 OOM，可降低 batch size、减少 cutoff_len，或改用 LoRA / QLoRA。

6.5 代码运行命令

单卡训练：

```
GPU_ID=0 bash dpo/scripts/train.sh
```

指定配置文件：

```
CONFIG=dpo/configs/qwen15_code_full_dpo.yaml GPU_ID=0 bash dpo/scripts/train.sh
```

八卡 torchrun 训练示例：

```
export CUDA_VISIBLE_DEVICES=0,1,2,3,4,5,6,7
export
PYTHONPATH=/data/yekaiyang/zjx/20260617/assignment_A/LlamaFactory/src:$PYTHONPATH
export HF_DATASETS_OFFLINE=1
export TRANSFORMERS_OFFLINE=1
export WANDB_DISABLED=true
export TOKENIZERS_PARALLELISM=false

torchrun --nproc_per_node=8 \
  -m llamafactory.cli train \
  dpo/configs/qwen15_code_full_dpo.yaml
```

评估 base 模型：

```
BATCH_SIZE=32 bash dpo/scripts/run_mbpp_base_eval.sh
```

评估 DPO 模型：

```
BATCH_SIZE=32 bash dpo/scripts/run_mbpp_dpo_final_eval.sh
```

并行评估 base 与 DPO 并生成对比指标：

```
BASE_GPUS=0 DPO_GPUS=1 BATCH_SIZE=16 bash dpo/scripts/run_mbpp_base_dpo_eval.sh
```

6.6 代码运行图例

```
[transformers] The tokenizer has new PAD/BOS/EOS tokens that differ from the model config and generation config. The model
config and generation config were aligned accordingly, being updated with the tokenizer's values. Updated tokens: {'bos_tok
en_id': None, 'pad_token_id': 151643}.
[INFO|trainer.py:1469] 2026-06-19 02:26:18,187 >> ***** Running training *****
[INFO|trainer.py:1470] 2026-06-19 02:26:18,187 >> Num examples = 8,924
[INFO|trainer.py:1471] 2026-06-19 02:26:18,187 >> Num Epochs = 3
[INFO|trainer.py:1472] 2026-06-19 02:26:18,187 >> Num update steps per epoch = 140
[INFO|trainer.py:1473] 2026-06-19 02:26:18,187 >> Instantaneous batch size per device = 1
[INFO|trainer.py:1476] 2026-06-19 02:26:18,187 >> Total train batch size (w. parallel, distributed & accumulation) = 64
[INFO|trainer.py:1477] 2026-06-19 02:26:18,187 >> Gradient Accumulation steps = 8
[INFO|trainer.py:1478] 2026-06-19 02:26:18,187 >> Total optimization steps = 420
[INFO|trainer.py:1479] 2026-06-19 02:26:18,188 >> Number of trainable parameters = 463,987,712
{'loss': '0.6934', 'grad_norm': 'inf', 'learning_rate': '1.19e-07', 'rewards/chosen': '9.879e-06', 'rewards/rejected': '0.0
005194', 'rewards/accuracies': '0.4688', 'rewards/margins': '-0.0005095', 'logps/chosen': '-236.8', 'logps/rejected': '-185
.3', 'logits/chosen': '-1.98', 'logits/rejected': '-1.983', 'epoch': '0.03584'}
{'loss': '0.6903', 'grad_norm': '37.09', 'learning_rate': '5.952e-07', 'rewards/chosen': '0.003406', 'rewards/rejected': '-
0.00242', 'rewards/accuracies': '0.6281', 'rewards/margins': '0.005826', 'logps/chosen': '-234.7', 'logps/rejected': '-172.
8', 'logits/chosen': '-1.97', 'logits/rejected': '-1.928', 'epoch': '0.07168'}
{'loss': '0.6426', 'grad_norm': '33.48', 'learning_rate': '1.19e-06', 'rewards/chosen': '0.04918', 'rewards/rejected': '-0.
05983', 'rewards/accuracies': '0.9062', 'rewards/margins': '0.109', 'logps/chosen': '-229.9', 'logps/rejected': '-183.1', '
logits/chosen': '-1.928', 'logits/rejected': '-1.91', 'epoch': '0.1075'}
{'loss': '0.5008', 'grad_norm': '20.33', 'learning_rate': '1.786e-06', 'rewards/chosen': '0.133', 'rewards/rejected': '-0.3
491', 'rewards/accuracies': '0.9406', 'rewards/margins': '0.4821', 'logps/chosen': '-222.5', 'logps/rejected': '-165.6', 'l
ogits/chosen': '-1.932', 'logits/rejected': '-1.985', 'epoch': '0.1434'}
{'loss': '0.3226', 'grad_norm': '14.81', 'learning_rate': '2.381e-06', 'rewards/chosen': '0.05747', 'rewards/rejected': '-1
.467', 'rewards/accuracies': '0.9312', 'rewards/margins': '1.524', 'logps/chosen': '-238.4', 'logps/rejected': '-202.7', 'l
ogits/chosen': '-1.971', 'logits/rejected': '-1.915', 'epoch': '0.1792'}
{'loss': '0.2656', 'grad_norm': '15.84', 'learning_rate': '2.976e-06', 'rewards/chosen': '-0.5335', 'rewards/rejected': '-2
.601', 'rewards/accuracies': '0.9862', 'rewards/margins': '2.157', 'logps/chosen': '-238.9', 'logps/rejected': '-206.1', 'l
ogits/chosen': '-1.933', 'logits/rejected': '-1.915', 'epoch': '0.2154'}
```



损失曲线

```
mbpp_metrics.json X
zjx > 20260617 > assignment_A > dpo > outputs > mbpp_eval_base > mbpp_metrics.json > ...
1  {
2    "benchmark": "MBPP",
3    "config": "sanitized",
4    "split": "test",
5    "model_path": "Qwen1.5-0.5B-Chat",
6    "predictions": "dpo/outputs/mbpp_eval_base/mbpp_generations.jsonl",
7    "num_tasks": 257,
8    "pass_at_1": 0.0038910505836575876,
9    "syntax_pass_rate": 0.7937743190661478,
10   "avg_test_pass_rate": 0.00902061855670103,
11   "passed_tasks": 1,
12   "total_tests": 776,
13   "passed_tests": 7,
14   "prompt": {
15     "source": "Google Research MBPP README",
16     "template": "You are an expert Python programmer, and here is your task: {prompt} Your code sho",
17     "mode": "zero_shot",
18     "example_task_ids": []
19   },
20   "execution_note": "Generated Python is executed in a temporary subprocess with timeout and basic",
21 }
```

Base 测试结果和指标

6.7 训练输出与示例结果

输出文件	说明
dpo/outputs/qwen15_code_full_dpo/model.safetensors	训练后的模型重。
dpo/outputs/qwen15_code_full_dpo/training_loss.png	LLaMA-Factory 自动绘制的训练 loss 曲线。
dpo/outputs/qwen15_code_full_dpo/training_rewards_accuracies.png	DPO 奖励准确率曲线。
dpo/outputs/qwen15_code_full_dpo/trainer_state.json	训练过程日志，包括 loss、reward、learning_rate 等。
dpo/outputs/qwen15_code_full_dpo/checkpoint-*	训练中间 checkpoint。
dpo/outputs/mbpp_eval_base/mbpp_metrics.json	base 模型 MBPP 执行评测结果。
dpo/outputs/mbpp_eval_dpo_final/mbpp_metrics.json	DPO 模型 MBPP 执行评测结果。

当前已完成的一次训练结果示例：

指标	数值
epoch	3.0
global_step	420
train_loss	0.0643
train_runtime	1272.64 秒
train_samples_per_second	21.037

当前 MBPP sanitized test 执行评测结果示例：

模型	pass_at_1	syntax_pass_rate	avg_test_pass_rate	passed_tasks
Base Qwen1.5-0.5B-Chat	0.0039	0.7938	0.0090	1
DPO qwen15_code_full_dpo	0.0156	0.7121	0.0361	4

说明：MBPP 评测会将生成代码写入临时文件，在子进程中运行 assert 测试，并设置超时与基础资源限制。因此它是执行型 benchmark，不只是文本相似度评估。

6.8 本节实训进阶要求

1. 将当前 full DPO 改为 LoRA DPO: 把 `finetuning_type` 改为 `lora`, 并增加 `lora_rank`、`lora_alpha`、`lora_dropout`、`lora_target` 等参数。
2. 对比 full finetuning 与 LoRA finetuning 的显存占用、训练速度、loss 曲线、MBPP `pass_at_1` 和样例质量。
3. 尝试 DPO 类损失函数改进, 例如 `pref_loss: hinge`、`ipo`、`orpo`、`simpo`, 并对比训练稳定性和最终评测结果。
4. 使用 SimPO 或 ORPO 作为 DPO 改进算法进行实验, 分析是否减少 `reference model` 依赖、是否降低显存占用。
5. 学习并尝试 KTO 偏好优化方法, 完成数据格式适配并运行 `stage: kto` 的训练配置。
6. 尝试 GRPO 或 PPO 等强化学习式对齐算法, 设计代码任务 `reward`, 例如语法正确、通过单元测试、函数名匹配、输出格式正确等。
7. 对比 Base、Full DPO、LoRA DPO、SimPO/ORPO、GRPO/PPO 等模型在训练成本和 MBPP 执行评测上的效果差异。

七、推理增强策略

7.1 模块目标

A5 模块不继续训练模型，而是在推理阶段通过 prompt 设计、CoT、采样、验证器重排序和 test-time scaling 方法提升模型输出质量。它适合与 A4 模块形成“训练增强 + 推理增强”的对比。

7.2 本节实训基础要求（可以在当前代码的引导下完全实现）

1. 分别加载训练前的 base 模型和训练后的 DPO 模型，对同一批代码任务进行生成推理。
2. 使用相同测试集、相同生成参数，对 base 模型和 DPO 模型输出进行公平对比。
3. 使用 CoT few-shot prompt 增强推理过程，引导模型输出 Reasoning、Key steps 和 Final code。
4. 从模型回答中提取 Final code 代码块，并进行语法检查和 MBPP assert 执行测试。
5. 将推理结果保存为 metrics 和 cases 文件，方便后续展示和错误分析。

7.3 CoT Prompt 设计

当前 CoT 模板位于 `tts/cot_code_example.py`。系统提示要求模型严格按照 Reasoning、Key steps、Final code 三段输出，评测时只提取 Final code 中的 Python 代码。

```
Reasoning:
- Briefly explain the idea.

Key steps:
1. List the implementation steps.

Final code:
```python
complete executable Python code here
```
```

7.4 代码运行命令

查看 CoT prompt 示例：

```
python3 tts/cot_code_example.py
```

评估 base 模型的普通 MBPP 结果：

```
BATCH_SIZE=32 bash dpo/scripts/run_mbpp_base_eval.sh
```

评估 DPO 模型的普通 MBPP 结果：

```
BATCH_SIZE=32 bash dpo/scripts/run_mbpp_dpo_final_eval.sh
```

评估 CoT 增强后的模型：


```
MODEL_PATH=Qwen1.5-0.5B-Chat \  
BATCH_SIZE=32 \  
OUTPUT_DIR=tts/outputs/cot_code_eval_base \  
bash tts/run_cot_code_eval.sh
```

评估 DPO 模型 + CoT:

```
MODEL_PATH=dpo/outputs/qwen15_code_full_dpo \  
BATCH_SIZE=32 \  
OUTPUT_DIR=tts/outputs/cot_code_eval_dpo \  
bash tts/run_cot_code_eval.sh
```

小样本快速调试:

```
MODEL_PATH=dpo/outputs/qwen15_code_full_dpo \  
BATCH_SIZE=4 \  
OUTPUT_DIR=tts/outputs/cot_code_eval_debug \  
bash tts/run_cot_code_eval.sh --limit 20
```

7.5 处理流程

1. 读取输入任务，默认使用 ../mbpp/sanitized/test-00000-of-00001.parquet。
2. 将每道题转换为包含题目描述和测试用例的 task 文本。
3. 调用 cot_code_example.py 中的 build_chat_messages 构造 CoT prompt。
4. 加载指定模型，按 batch 生成回答。
5. 从回答中提取 Final code 代码块。
6. 使用 ast.parse 做语法检查。
7. 将代码和 MBPP 测试用例写入临时 Python 文件，在子进程中执行。
8. 统计 syntax_pass_rate 、 pass_at_1 、 avg_test_pass_rate 、 exact_match 和 avg_code_token_f1。

7.6 输出文件

| 文件 | 说明 |
|---|-----------------------|
| tts/outputs/cot_code_eval/cot_code_metrics.json | CoT 评测指标。 |
| tts/outputs/cot_code_eval/cot_code_cases.jsonl | 每题任务、回答、代码、测试结果。 |
| tts/outputs/cot_code_eval_base | base 模型 + CoT 输出目录示例。 |
| tts/outputs/cot_code_eval_dpo | DPO 模型 + CoT 输出目录示例。 |

7.7 本节实训进阶要求：Test-Time Scaling

Test-Time Scaling 的核心思想是：训练结束后不再更新模型参数，而是在推理阶段投入更多计算量，通过多候选采样、搜索、验证、执行反馈和反思修正来选择更优答案。

1. 实现 Self-Consistency 多路径采样：同一道题采样多个推理路径，提取最终代码或执行结果，选择最一致的答案。

2. 实现 Best-of-N 生成与重排序：同一道题生成 N 个候选代码，根据语法、测试通过数、代码格式、verifier 分数选择最优答案。

3. 实现 Reflexion 反思修正机制：根据语法错误或测试失败信息生成反思，再带着反思重新生成代码。

4. 实现 Tree of Thoughts 搜索式推理：生成多个中间思路，对路径进行评分、扩展和剪枝，最后选择得分最高的代码。

5 系统对比 greedy decoding、CoT、Self-Consistency、Best-of-N、Tree of Thoughts 的效果与成本。

| 策略 | 需要修改的代码 | 主要指标 |
|------------------|---|---------------------|
| Self-Consistency | tts/evaluate_cot_code.py 增加 num_samples | 多数一致率、pass_at_1、耗时 |
| Best-of-N | 增加候选生成与规则打分函数 | best pass_at_1、生成成本 |
| Reflexion | 增加多轮 refine prompt | 修正前后通过率 |
| Tree of Thoughts | 增加 tree_width / tree_depth 搜索 | 搜索成本、最终 pass rate |